CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 4th October, 2026)

Due Date: 12th October, 2026 (By 1:00 pm)

For Batch - R1

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. Evaluation will be done during 12th October to 14th October 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
5. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
6. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*

Urban Company Service Management System

Urban Company connects customers with professionals who provide home services such as cleaning, plumbing, appliance repair, beauty services, and electrical work. Design a linked-list-based system to manage customers, professionals, services, and service requests while keeping the representation compact and the required operations efficient.

You are free to decide which type of LinkedList you would like to use, whether Singly Linked List, Doubly Linked List or Circular Linked List.

Data

The system maintains the following entities as records :

1. Customer

- Customer ID
- Name
- Pincode
- Contact Number

2. Professional

- Professional ID
- Name
- Pincode
- Contact Number
- Availability status
- Services provided by the professional

3. Service

- Service ID
- Service Name

- Base Price

4. Service Request

- Request ID
- Customer
- Service
- Assigned Professional (if any)
- Requested Date and Time
- Status: Pending, Assigned, Completed, or Cancelled

A professional can provide multiple services, but **each professional can provide at most 5 different services**.

A customer may raise multiple service requests. A professional may handle multiple requests over time, but can handle only one request at a time.

Functions : Implement the following operations:

1. Register Customer

Add a new customer to the system.

2. Register Professional

Add a new professional. The professional initially provides one or more services and is marked as available.

3. Add Service

Add a new service with its base price.

4. Create Service Request

A customer requests a particular service.

- Verify that the customer and service exist.
- Search for an available professional who provides the requested service.
- Prefer a professional from the same pincode.
- If none is available in the same pincode, select an available professional from another pincode.
- If no suitable professional is available, keep the request pending.
- Once assigned, the professional becomes unavailable.

5. Complete Service

Complete an assigned request.

- Mark the request as completed.
- Make the professional available again.
- The newly available professional should be considered for an appropriate pending request, if one exists.

6. Cancel Service Request

Cancel a pending or assigned request.

- Mark the request as cancelled.
- If a professional was assigned, make the professional available again.
- The freed professional should be considered for an appropriate pending request.

7. Search Operations

Implement:

a. Given a Service ID, display all available professionals who provide that service.
b. Given a Professional ID, display all services provided by that professional.
c. Given a Customer ID, display all service requests made by that customer in chronological order.

8. Sorting Operations

a. Service Demand:
Display all services in decreasing order of the number of service requests made for each service.

b. Professional Activity:
Display all professionals in decreasing order of the number of requests they have successfully completed.

Do not store these counts as additional fields in the nodes. Derive them from the representation maintained by your system.

9. Display Pending Requests

Display all currently pending requests in the order in which they were created.

10. Display Service History

Given a Professional ID, display the requests completed by that professional in chronological order.

Requirements and Constraints

1. The primary collections of **Customers, Professionals, Services, and Service Requests must be implemented using linked lists**.
2. **Do not create duplicate copies of an entity.** For example, there must be only one actual node representing a particular customer, professional, service, or request.
3. The maximum of **5 services per professional** is a property of the system and should be considered while designing the representation.
4. Do not store derived information such as *numberOfRequests*, *totalCompleted*, or *serviceDemand* as additional node fields. These values must be calculated from the maintained data.
5. Input data will not be provided in sorted order.
6. Initially sort the data based on IDs, for storage.
7. Your design should avoid unnecessary duplication of relationship information and should aim to minimize additional storage while keeping the required operations efficient.
8. Handle cases such as:
    - duplicate IDs,
    - invalid customer/service/professional IDs,
    - no available professional,
    - cancellation of an already completed/cancelled request,
    - completion of an unassigned request,
    - professional with no pending request after becoming available.
9. Implement the system using a menu-driven program and demonstrate all operations with suitable test data.

CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 4th October, 2026)

Due Date: 12th October, 2026 (By 1:00 pm)

For Batch – R2

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. Evaluation will be done during 12th October to 14th October 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
5. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
6. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*

Traffic Violation & Fine Management System

Implement a traffic violation and fine management system for a city traffic police department.

Maintain a linked list of registered vehicles, where each vehicle stores

- registration number,
- owner name,
- contact details,
- address,
- total fines accrued,
- total fines paid,
- pending amount, and
- number of violations committed so far.

Each violation issued against a vehicle should be recorded in a separate linked list of violations.

A violation has a

- violation ID,
- the vehicle it belongs to,
- the type of violation (illegal parking, over-speeding, red-light jump, or helmet/seatbelt infraction),
- the location where it occurred,
- the date and time, the fine amount, and
- whether it has been paid or is still pending.

Maintain a separate linked list of enforcement locations (e.g., Aath-Rasta Road Junction, Sitabuldi Market), each identified by a location ID and name. Violations should reference a location from this list rather than storing the location as free text.

Payments made against violations should be recorded in a linked list (implemented as a stack), storing the transaction ID, the violation it was paid against, the amount paid, the mode of payment, and the date.

You are free to decide which type of LinkedList you would like to use, whether Singly Linked List, Doubly Linked List or Circular Linked List.

Implement the following operations:

1. When a new vehicle is registered, its details should be added to the vehicle list in sorted order based on the registration number.
2. When a violation is reported against a vehicle, the fine should be calculated based on the violation type. If the vehicle has more than 2 past violations, an additional surcharge should be applied to the fine. The violation should be added to the violation list, and the vehicle's total fine accrued, pending amount, and violation count should be updated accordingly.
3. When a payment is made against a violation, the violation should be marked as paid, the vehicle's pending amount should be reduced, and the payment should be recorded in the payment history.
4. Given an owner's name, find the vehicle(s) belonging to that owner and list all violations committed by those vehicles, along with their type, location, date, fine amount, and payment status.
5. Given a location, find all violations that occurred there and list the owner name and contact details for each vehicle involved.
6. Sort and display all vehicles in descending order based on the number of unpaid violations they currently have.
7. Sort and display all locations in descending order based on how many violations have occurred there.
8. Sort and display all locations in descending order based on the total fine revenue generated at that location.
9. Sort and display all violation types in descending order based on how frequently they occur across the city.
10. For each violation type, find what fraction of those violations were committed by repeat offenders (vehicles with more than 2 violations), and display violation types ranked by this proportion.
11. Given a threshold number, find and display all vehicles whose violation count is greater than or equal to that threshold, along with the owner's details.
12. Given a vehicle's registration number, display all of its pending (unpaid) violations along with their details.
13. Sort and display all vehicles in descending order based on their pending fine amount.
14. Display the complete payment history in reverse chronological order, showing the transaction ID, violation paid, amount, and mode of payment.

Note:

- The number of vehicles and violations will not exceed 1000, and the number of locations will not exceed 50.
- Assume initial data (vehicles, locations, and violations) is generated through file handling and is unsorted; sorting should happen when the data is read or when a display/ranking operation is requested.
- Implement the lists using an appropriate choice of linked list (single, double, or circular) for each entity.
- Use stack and queue ADTs wherever appropriate.

CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 4th October, 2026)

Due Date: 12th October, 2026 (By 1:00 pm)

For Batch – R3

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. Evaluation will be done during 12th October to 14th October 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
5. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
6. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*

DigiYatra Flight Check-in and Boarding Management System

DigiYatra handle the flight check-in and boarding of flights, passengers, check-in, boarding, seat allocation, and flight-status updates continuously. Design a linked-list-based system to manage flights and passengers while keeping the representation compact and ensuring that operations on one list correctly update the related information in the other list.

You are free to decide which type of LinkedList you would like to use, whether Singly Linked List, Doubly Linked List or Circular Linked List.

Data Structures : The system maintains the following entities:

1. Flight

- Flight ID
- Flight Number
- Source
- Destination
- Departure Time
- Arrival Time
- Total Seats
- Available Seats
- Flight Status

2. Passenger

- Passenger ID
- Name
- Age

- Gender
- Contact Number
- Flight ID
- Seat Number
- Ticket Type: Economy, Business, or First Class
- Check-in Status
- Boarding Status

3. Check-in Queue

- Passenger ID
- Passenger reference
- Queue links

4. Boarding Queue

- Passenger ID
- Passenger reference
- Ticket Type / priority
- Queue links

5. Boarding History

- Passenger ID
- Passenger Name
- Flight ID
- Seat Number
- Boarding Time

Flights and passengers must be maintained as separate linked lists. A Passenger stores the Flight ID as a reference to the corresponding Flight rather than duplicating complete flight information. The Flight node maintains Available Seats, while passenger records determine passenger-related counts when required.

Functions

1. Add Flight: Add a new flight to the Flight Linked List. Reject duplicate Flight IDs. Store flights in ascending order of Departure Time.

2. Delete Flight: Delete a flight only if no passenger currently refers to that Flight ID. This operation must search both the Flight List and Passenger List before deletion.

3. Search and Display Flight: Given a Flight ID, Flight Number, Source, or Destination, display the matching flight. Also derive and display the number of passengers, checked-in passengers, boarded passengers, and available seats using the Passenger List.

- Display sorted based on Departure Time
- Display sorted based on Source
- Display sorted based on Destination
- Display sorted first on Source and, for the same source, based on Departure Time

4. Add Passenger: Add a passenger to the Passenger Linked List. Verify that the Passenger ID is unique, the Flight ID exists, the flight is accepting passengers, and a seat is available. Allocate a seat and update Available Seats in the corresponding Flight node.

- The flight exists.
- Seats are available.
- The PassengerID is unique.
- Check whether seats are available.
- Allocate a seat.
- Update AvailableSeats in the corresponding Flight node.
- Insert passenger into the Passenger List.

A seat should be automatically allocated to the passenger.
SS

5. Delete Passenger: Delete a passenger after finding the corresponding Flight. Release the passenger's seat and increase Available Seats in the Flight node.

6. Board Passenger: Process the next passenger from the Boarding Queue. A passenger can board only after successful check-in and only when the flight is in the Boarding state. Update Boarding Status and add the passenger to Boarding History.

7. Cancel Flight: Cancel a selected flight. A cancelled flight cannot accept new passengers or boarding operations.

8. Seat Reallocation: For a selected flight, rearrange passenger seats according to Ticket Type priority: First Class, Business, then Economy. Maintain consistency between the Passenger List and Flight List.

9. Real-Time Flight Analysis: Display the most crowded flight, flight with the highest occupancy percentage, most popular destination, destination with the maximum First-Class passengers, flight with the highest senior-passenger percentage, and flight with maximum available seats.

Flight Status

- Scheduled
- Check-in Open
- Boarding
- Departed
- Cancelled

Priority Rules

- First Class passengers are processed before Business Class passengers.
- Business Class passengers are processed before Economy passengers.
- Passengers having the same ticket priority preserve their check-in/arrival order.
- A passenger cannot board before check-in.
- A passenger cannot board a cancelled or departed flight.

Requirements and Constraints

1. The primary collections of Flights and Passengers must be implemented using linked lists.
2. Flights and Passengers should use Doubly Linked Lists.
3. The Check-in Queue must be implemented using a linked list.
4. The Boarding Queue must be implemented using linked nodes with priority handling.
5. Boarding History may be implemented using a Singly Linked List or linked Stack.
6. Do not create duplicate copies of a flight or passenger entity. There must be only one actual node representing each entity.

7. Do not store derived information such as passengerCount, checkedInCount, boardedCount, occupancyPercentage, or destinationDemand as additional node fields. Derive these values from the maintained lists.
8. Passenger records should refer to flights using Flight ID instead of duplicating complete flight information.
9. Input data will not be provided in sorted order. Initially organize the linked lists according to the required storage order.
10. Handle duplicate IDs, invalid Flight IDs, invalid Passenger IDs, full flights, unavailable seats, cancelled flights, departed flights, empty queues, duplicate check-in, and invalid boarding attempts.
11. When an operation modifies a passenger's flight or seat, the corresponding Flight node must also be updated.
12. Implement the system using a menu-driven program and demonstrate all operations with suitable test data.

CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 4th October, 2026)

Due Date: 12th October, 2026 (By 1:00 pm)

For Batch – R4

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. Evaluation will be done during 12th October to 14th October 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
5. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
6. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*

Mining for the Gold

Consider a gold mining company that operates several mining sites and employs miners who extract gold using different mining methods (underground mining or open-pit/alluvial mining). When a mining site requires labour, the site supervisor sends a request through the company's system, optionally specifying a preferred mining method (underground or open-pit) or leaving it as "any". As soon as a request is generated, the location of the mining site is recorded and the system searches for all nearby free miners (of the preferred type, or any if not specified). The nearest free miner is assigned to the site. The system must maintain complete information about miners, gold buyers, transaction history, and real-time status (availability) of all miners.

You are required to implement this system using linked lists in C language. All data must be stored dynamically using structures and pointers. The system must support at least the functionalities listed below.

You are free to decide which type of LinkedList you would like to use, whether Singly Linked List, Doubly Linked List or Circular Linked List.

Data Structures:

You must create the following linked lists:

1. **Miner List** — Stores details of all miners (both underground and open-pit) working for the company. Each node contains:

- m_ID (unique integer)
- name (string)
- mining_type (integer: 0 for underground, 1 for open-pit)
- current_location (structure with x and y coordinates, integers) — location of the mining site the miner is stationed near
- status (integer: 0 = free, 1 = assigned)
- total_earnings (float) — can be updated after each completed extraction
- next pointer

2. **Buyer List** — Stores details of all gold buyers who have ever purchased gold from the company. Each node contains:

- b_ID (unique integer)
- name (string)
- mobile_no (string)
- frequency (integer) — total number of purchases made by this buyer
- next pointer

3. **Transaction History List** — Stores every completed gold extraction and sale. Each node contains:

- transaction_id (unique integer)
- m_ID (miner ID)
- b_ID (buyer ID)
- mining_type (0/1) — useful for quick analysis
- gold_quantity (float, in grams) — quantity of gold extracted and sold
- amount (float, calculated as gold_quantity × gold_rate)
- date (optional, can be simulated by an integer timestamp)
- next pointer

Functions to Implement:

You must implement the following functions. You can add more if needed.

4. **void addMiner(Miner **head, int id, char name[], int type, int x, int y)** — Adds a new miner at the end of the Miner list. The miner is initially free (status = 0) and total_earnings = 0. Coordinates of the site nearest to the miner are given as input.

5. **void addBuyer(Buyer **head, int id, char name[], char mobile[])** — Adds a new buyer at the end of the Buyer list with frequency = 0. If a buyer with the same id or mobile already exists, do not add a duplicate (you may simply return or update nothing).

6. **Miner* findNearestMiner(Miner *mHead, int site_x, int site_y, int prefType)** — Finds the nearest free miner (status 0) of type prefType (0 for underground, 1 for open-pit, -1 for any) within a 5 km radius from the mining site's location (site_x, site_y). Use Euclidean distance: distance = sqrt((x1-x2)^2 + (y1-y2)^2). Return a pointer to that miner node. If no miner is available within 5 km, return NULL. Note: For simplicity, assume coordinates are in kilometres and no conversion is needed.

7. **int requestMining(Miner *mHead, Buyer *bHead, Transaction **tHead, int b_id, int site_x, int site_y, int prefType)** — Handles a new mining request.

- Verify that the buyer exists (by b_id) in the Buyer list. If not, print an error and return -1.
- Call findNearestMiner() to obtain the nearest free miner of the preferred type.
- If a miner is found, create a new transaction node with a unique transaction_id (you can maintain a static counter). Set gold_quantity = 0 and amount = 0 initially.
- Change the miner's status to 1 (assigned).
- Append the transaction to the Transaction History list.
- Return the new transaction_id.
- If no miner is found, return -1.

8. **void completeMining(Miner *mHead, Buyer *bHead, Transaction *tHead, int transaction_id, float quantity)** — Completes a mining and sale operation.

- Find the transaction with the given transaction_id in the Transaction list.

- Update its gold_quantity with the provided quantity (in grams).
- Calculate amount based on mining type and the current gold rate:
  - Underground (type 0): ₹6500 per gram (higher-purity ore)
  - Open-pit/alluvial (type 1): ₹5800 per gram
- Add this amount to the miner's total_earnings (update the miner node).
- Increment the buyer's frequency by 1.
- Set the miner's status back to 0 (free).
- (Optionally, you may also store the completion time.)

9. **float calculateMinerEarnings(Miner *mHead, int m_id)** — Computes the total earnings of a specific miner by summing the amount of all transactions associated with that miner in the Transaction list. Alternatively, you could use the total_earnings field if you keep it updated — but this function demonstrates traversal and summation.
10. **void displayTopMiners(Miner *mHead, Transaction *tHead)** — Displays the names of the three most successful miners based on their total earnings.

- Compute earnings for each miner (using calculateMinerEarnings or the stored field).
- Sort miners by earnings (or find top three).
- Print their names and total earnings.

11. **void displayFrequentPairs(Miner *mHead, Buyer *bHead, Transaction *tHead)** — Finds the buyer–miner pair (i.e., a specific (m_ID, b_ID)) that has transacted together the most number of times.

- Count occurrences of each pair from the Transaction list.
- Retrieve the names of the miner and buyer from the respective lists.
- Display their names and the number of transactions they share.

12. **void displayAvailableMiners(Miner *mHead)** — Prints the details (ID, name, mining type, current location) of all miners whose status is free (0).

- Clearly indicate whether it's an underground miner or an open-pit miner.

13. **void updateMinerLocation(Miner *mHead, int m_id, int new_x, int new_y)** — Updates the current coordinates of a miner (simulating relocation to a new site).

- Find the miner by m_id and set his current_location to (new_x, new_y).
- If the miner is assigned, you may optionally prevent location update or allow it — specify your choice.

14. **void deleteMiner(Miner **mHead, int m_id)** — Removes a miner from the Miner list when they leave the company.

- Ensure the miner is currently free (status 0). If assigned, print an error and do not delete.
- Free the node and adjust links.

15. **void displayTransactionHistory(Transaction *tHead)** — Prints all transactions with details: transaction ID, miner ID, buyer ID, mining type, gold quantity, amount.

Additional Notes

- All lists must be implemented as singly linked lists.
- Use appropriate typedef for structures.
- For simplicity, assume coordinates are integers representing kilometres. The 5 km radius check must use Euclidean distance (you may use sqrt from math.h; remember to link with -lm).
- The gold rates are fixed: Underground = ₹6500/gram, Open-pit/alluvial = ₹5800/gram.
- All functions must handle edge cases gracefully (e.g., empty lists, non-existent IDs, no miner found, etc.) and print appropriate messages.

- You are free to design additional helper functions (e.g., for sorting, searching) but the above 12 (or at least 10) must be present and correctly implemented.
- The program should provide a menu-driven interface to test all functionalities. The user should be able to:
    - Add a miner
    - Add a buyer
    - Request mining (assign a miner to a site)
    - Complete mining (record extraction and sale)
    - Display top miners
    - Display frequent pairs
    - Display available miners
    - Update miner location
    - Delete a miner
    - Display transaction history
    - Exit

CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 4th October, 2026)

Due Date: 12th October, 2026 (By 1:00 pm)

For Batch – R5

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. Evaluation will be done during 12th October to 14th October 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
5. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
6. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*

Smart Waste Collection Management

Modern cities increasingly rely on IoT-enabled smart bins that continuously report their fill level, so that garbage trucks are dispatched only when and where they are actually needed, instead of following a fixed daily route. You are tasked with building a Smart Waste Collection Management System that simulates this behavior: bins report fill levels, the system automatically raises a collection request once a bin crosses its threshold, the nearest available collection vehicle is dispatched, and once the pickup is done, the bin resets and vehicle/driver statistics are updated.

Implement this application using linked lists. You are free to decide which type of LinkedList you would like to use, whether Singly Linked List, Doubly Linked List or Circular Linked List.

1. Nodes

1.1 Waste Bin Node

- Bin ID (unique integer)
- Zone / Area name
- Location coordinates (x, y)
- Current fill level (0–100%) and threshold (default 75%)
- Status: Normal / Awaiting-Collection / Under-Collection

1.2 Collection Vehicle Node

- Vehicle ID (unique integer)
- Vehicle capacity and current load (in kg)
- Current location (x, y)
- Status: Free / On-Collection / At-Depot
- Driver ID (reference to Driver Node)

1.3 Driver Node

- Driver ID (unique integer)

- Name
- Assigned Vehicle ID (reference to Vehicle Node)
- Total collections handled
- Performance rating (derived — see Section 3)

1.4 Collection Request Node

- Request ID (unique integer)
- Bin ID (reference to Waste Bin Node)
- Priority: High (fill level ≥ 90%) Otherwise Normal
- Assigned Vehicle ID (NULL until assigned)
- Status: Pending / Assigned / Completed (Completed requests double as collection history no separate list needed)

2. Functions to Implement

a. **RegisterBin():** Register a new bin with its zone, location, capacity and threshold bins are stored sorted by Bin ID.

b. **UpdateFillLevel():** Update a bin's fill level if it crosses the threshold, automatically call GenerateCollectionRequest() (skip if the bin already has a Pending/Assigned request).

c. **GenerateCollectionRequest():** Create a request for a bin that crossed its threshold, assign it a priority, and insert it into the pending queue (High before Normal, oldest-first within a priority).

d. **AssignNearestVehicle():** Take the next request from the pending queue and assign the nearest Free vehicle (Euclidean distance) with enough remaining capacity; mark vehicle On-Collection, request Assigned, bin Under-Collection.

e. **CompleteCollection():** Record waste collected, reset the bin, update the vehicle's load/totals and the driver's collection count, free the vehicle (or send it At-Depot if full), and mark the request Completed.

f. **RankVehicles(byField):** Sort and display vehicles in descending order by a chosen metric — total waste collected.

g. **Get_Driver_Performance(DriverID):** Print a driver's total collections, total waste collected, and average waste collected per trip.

h. **Display_Pending_Requests():** Display the pending-requests queue in the order it will be served (High priority first, then Normal, oldest-first within each).

i. **Find_Overflowing_Bins():** Scan all bins and list every bin whose fill level has crossed its threshold but does not yet have an Assigned request an alert list for bins that slipped through.

j. RankDriversForAVehicle(VehicleID) : It is possible that the same vehicle is driven by different drivers at different times. What changes will you make to the data structures for such case ? Assuming such changes are made (by defining some newer structures), write a function that will sort the list of drivers assigned to givem VehicleID based on their overall performance.

3. Policies

3.1 Threshold & Priority Policy

- A bin's threshold is configurable at registration (default 75%).
- A request is raised as soon as fill level reaches/crosses the threshold.
- Priority is High if fill level ≥ 90%, otherwise Normal.
- High priority requests are always served before Normal. FIFO within the same priority.

3.2 Vehicle Allocation Policy

- The nearest Free vehicle with enough remaining capacity is assigned. if not eligible, the next-nearest is checked, and so on.
- If no eligible vehicle is found, the request stays Pending and is retried when a vehicle next becomes Free.
- A vehicle whose load reaches capacity must go At-Depot to offload (load resets to 0, status becomes Free) before its next assignment.

3.3 Driver Performance Policy

- A driver's performance rating is derived from (total waste collected ÷ total collections).

Note

- Bins ≤ 200, vehicles/drivers ≤ 15 each, requests ≤ 500.
- Bin ID, Vehicle ID, Driver ID and Request ID are unique integers (1–1000).
- Assume 10 simulated days for testing collection history and reports.
- Create an initial dataset with at least 40 bins across 5–6 zones, 8–10 vehicles, and 8–10 drivers, plus at least 30 initial requests loaded unsorted via file handling, then sorted on read.
- Bin, Vehicle, Driver and Request details should each be stored as a linked list. choose single/double/circular based on the operations needed, and justify the choice in your report.
- Use stack/queue ADTs where appropriate for eg: a priority queue (via linked list) for the pending-requests list, and a stack for a driver's most-recent-collections log.